INACAP · PROGRAMACIÓN SEGURA

CartaPnk

Documentación técnica

Frontend y backend separados · Node.js · PostgreSQL · Docker · AWS · Cloudflare

Dato	Detalle
Sitio	https://cartapnk.xyz
Repositorio	https://github.com/Alkr4/CartaPnk
Versión del documento	1.0 — 10 de noviembre de 2026
Integrante(s)	
Docente	

Índice

1. Resumen 4
1.1 Qué cubre este documento 4
1.2 Estado actual 4
2. Arquitectura 5
2.1 Flujo de una petición 5
2.2 Componentes 5
2.3 Estructura del repositorio 5
3. Backend (API) 6
3.1 Arquitectura MVC 6
3.2 API REST 6
3.3 Modelo de datos 7
3.4 Panel de administración 7
4. Frontend 8
4.1 Cómo se evita el XSS 8
4.2 Nginx 8
5. Seguridad implementada 8
5.1 Hallazgos del informe de pentesting y cómo se resuelven 8
5.2 Controles del panel y de las imágenes 9
5.3 Contenedores y red 10
6. Protección de /admin: defensa en profundidad 10
6.1 Por qué importan las capas 5, 6 y 7 10
6.2 Por qué no están aplicadas (decisión consciente) 11
6.3 Cómo se activarían (para cuando se necesite) 11
7. Ambientes y Docker 12
7.1 Puesta en marcha 12
7.2 Variables de entorno principales 12
8. Flujo de Git, integración continua y Jenkins 13
8.1 Ramas y ambientes 13
8.2 Integración continua (GitHub Actions) 13
8.3 Plan de Jenkins con OWASP y MITRE (4.ª entrega) 13
9. Despliegue en AWS 14
9.1 Qué se crea 14
9.2 Comandos de `scripts/desplegar.sh` 15
9.3 Desplegar por primera vez 15
9.4 Demostrar el Auto Scaling 15
9.5 Actualizar la aplicación 16
10. Dominio, HTTPS y Cloudflare 16

10.1 Pasos que se hacen una sola vez (manuales) 16
10.2 Pasos por cada cuenta de AWS 16
10.3 Qué configura el comando cloudflare 16
10.4 Token de la API de Cloudflare 17
10.5 Diagnóstico de errores de Cloudflare 17
11. Operación y recuperación ante desastres 17
11.1 Qué ocurre si se borra algo 17
11.2 Reinstalación completa (~25 minutos) 18
11.3 Secretos 18
11.4 Solución de problemas 18
12. Pruebas y calidad 19
13. Limitaciones conocidas y trabajo pendiente 19
13.1 Limitaciones (laboratorio) 19
13.2 Pendiente 20

1. Resumen

CartaPnk es un menú digital para restaurantes. Es la reescritura de la aplicación PHP/MySQL original (`VulnerablePage`, usada en el laboratorio de explotación de vulnerabilidades) con una arquitectura moderna y segura: **frontend y backend separados**, API REST en **Node.js + Express**, base de datos **PostgreSQL**, contenedores **Docker** y despliegue escalable en **AWS** con dominio y HTTPS propios. La apariencia del sitio es la misma que la de la versión original.

Elemento	Detalle
Sitio público	https://cartapnk.xyz
Panel de administración	https://cartapnk.xyz/admin (solo el administrador; el cliente no ve ningún login)
Repositorio	https://github.com/Alkr4/CartaPnk (ramas `main` = producción, `develop` = QA)
Frontend	HTML, CSS y JavaScript sin frameworks, servido por Nginx
Backend	Node.js 24 LTS, Express 5, API JSON bajo `/api`, arquitectura MVC
Base de datos	PostgreSQL 17 (Amazon RDS en producción)
Imágenes	Amazon S3 (bucket privado) o disco local según el ambiente
Infraestructura	AWS: VPC, RDS, Application Load Balancer, Auto Scaling, S3, ECR, ACM; Cloudflare como DNS/CDN/WAF
Calidad	136 pruebas del backend (Jest + Supertest) y 16 del frontend (node:test + jsdom); ESLint con eslint-plugin-security

1.1 Qué cubre este documento

Este informe reúne en un solo lugar la documentación técnica del proyecto: arquitectura, seguridad, ambientes, despliegue en AWS, dominio y Cloudflare, operación y recuperación. Reemplaza a las guías sueltas del repositorio (`docs/*.md`), que repetían partes de lo mismo.

1.2 Estado actual

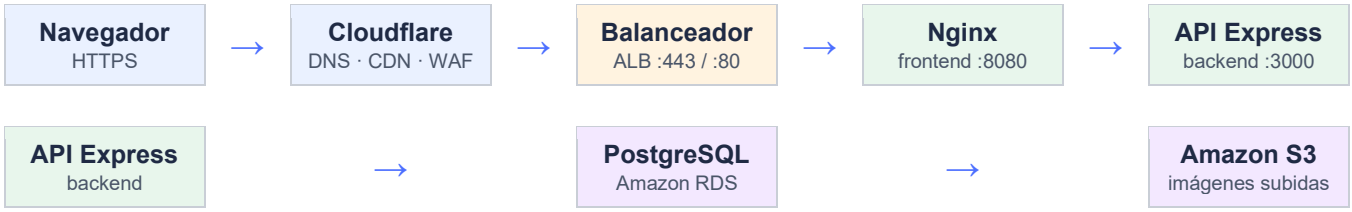
Área	Estado
Aplicación (carta, carrito, panel de administración)	**Terminada y desplegada** en producción
Separación frontend / backend	**Terminada y desplegada** (Nginx + API)
Tres ambientes (desarrollo, QA, producción) con Docker	**Terminados** y probados
AWS: RDS, balanceador, Auto Scaling, S3	**Desplegado y funcionando**
Dominio propio con HTTPS de extremo a extremo	**Funcionando** (cartapnk.xyz)
Automatización de Cloudflare por API	Implementada (rama `feature/cloudflare-api`, pendiente de fusionar)
Integración continua en GitHub Actions	**Funcionando** (3 trabajos)
Pipeline de Jenkins con OWASP / MITRE (4.ª entrega)	Diseñado y con borrador de `Jenkinsfile`; **no ejecutado todavía**

Área	Estado
Protección adicional de /admin en Cloudflare (WAF, rate limiting, Access)	Documentada; **no aplicada a propósito** (ver sección 6)

2. Arquitectura

2.1 Flujo de una petición

El navegador **solo habla con Nginx** (mismo origen). Nginx entrega las páginas estáticas y reenvía `/api`, `/media` y `/salud` a la API, que no publica ningún puerto. Por eso no se necesita CORS y la cookie de sesión sigue siendo `HttpOnly` + `SameSite=Lax`.



2.2 Componentes

Componente	Responsabilidad	Tecnología
Frontend (`frontend/`)	Páginas y lógica del navegador: carta, carrito y panel de administración. Cabeceras de seguridad (CSP) y proxy hacia la API.	HTML + CSS + JavaScript puro, Bootstrap 5, jQuery y Owl Carousel (los del sitio original); Nginx `nginx-unprivileged`
Backend (`backend/`)	API REST: datos, sesión, autenticación por rol, validación, subida de imágenes.	Node.js 24, Express 5, `pg`, `express-session`, `helmet`, `multer`, `bcryptjs`
Base de datos	Datos de negocio, sesiones y contadores del límite de login (así la aplicación no guarda estado).	PostgreSQL 17
Almacén de imágenes	Fotos de restaurantes y platillos subidas desde el panel.	Amazon S3 (privado) o disco local
Infraestructura (`infra/`)	Red, base de datos, balanceador, Auto Scaling y bucket como código.	AWS CloudFormation
Automatización (`scripts/desplegar.sh`)	Un solo script para construir, subir, desplegar, configurar el dominio y destruir.	Bash + AWS CLI + API de Cloudflare

2.3 Estructura del repositorio

```
backend/    API (Express, MVC): src/  db/  tests/  tools/  Dockerfile
frontend/   Sitio web: public/ (HTML, CSS, JS, imágenes)  nginx/  tests/  Dockerfile
infra/      Plantilla de CloudFormation (VPC, RDS, ALB, Auto Scaling, S3)
scripts/    desplegar.sh (despliegue y operación en AWS)
docs/       Este informe y guías de apoyo
env/        Plantillas de configuración por ambiente (*.env.example)
docker-compose*.yml   Los tres ambientes        .github/workflows/ci.yml   Integración continua
```

3. Backend (API)

3.1 Arquitectura MVC

Cada petición atraviesa `routes` → `middlewares` → `controllers` → `models` → PostgreSQL y vuelve como JSON. La "vista" la dibuja el frontend.

Carpeta (`backend/src/`)	Responsabilidad
`models/`	Acceso a datos. **Es el único lugar con SQL** y siempre con parámetros (`$1, $2…`). Usa borrado lógico (`eliminado`) y transacciones donde se escriben varias tablas.
`controllers/`	Casos de uso: validan la entrada, llaman a los modelos y responden JSON.
`routes/`	Mapa URL → controlador bajo `/api`, más `/media` (imágenes) y `/salud`.
`middlewares/`	Seguridad transversal: cabeceras, sesión, CSRF, límites de intentos, autorización por rol y manejo de errores.
`storage/`	Imágenes subidas: validación por firma y almacén local o S3.
`config/`	Configuración por ambiente leída de variables de entorno (sin secretos en el código).
`utils/`	Validación por esquemas (lista blanca de campos), formato y URLs de fotos.

3.2 API REST

Todas las respuestas son JSON. Los errores tienen la forma `{ "error": "mensaje" }` y los de validación (422) añaden `errores: { campo: mensaje }`. Las peticiones que cambian datos exigen el token CSRF en la cabecera `X-CSRF-Token`.

Pública

Método y ruta	Descripción
`GET /api/sesion`	Estado de la sesión (usuario o nulo) y token CSRF.
`GET /api/restaurantes/primero`	Id del primer restaurante activo (la página de inicio muestra ese).
`GET /api/restaurantes/:id`	Carta completa: restaurante, destacados, cartas > categorías > platillos y comentarios.
`GET /api/carrito`	Productos de la sesión y datos del restaurante.
`POST /api/carrito/items`	Agrega un platillo por id. El precio y el nombre salen de la base de datos, nunca del cliente.
`DELETE /api/carrito/items/:posicion` · `DELETE /api/carrito`	Quita un producto o vacía el pedido.
`GET /media/…` · `GET /salud`	Imágenes subidas (solo claves con formato exacto) · salud para el balanceador (consulta la base de datos).

Administración (rol admin)

Método y ruta	Descripción
`POST /api/admin/login` · `POST /api/admin/logout`	Acceso y cierre de sesión. El login responde igual ante usuario inexistente, contraseña errónea o rol distinto de admin.
`/api/admin/restaurantes` (GET, POST) y `/restaurantes/:id` (GET, PUT, DELETE)	CRUD de restaurantes con su dirección e imagen.
`/api/admin/restaurantes/:id/cartas`, `/cartas/:id`	CRUD de cartas.
`/api/admin/cartas/:id/categorias`, `/categorias/:id`	CRUD de categorías.
`/api/admin/categorias/:id/items`, `/items/:id`	CRUD de platillos (precio, descripción, puntuación, destacado, imagen).
`DELETE /api/admin/comentarios/:id`	Moderación de comentarios.

3.3 Modelo de datos

Tabla	Contenido
`direcciones`, **`restaurantes`**	Datos del restaurante (RUT, nombre, contacto, foto) y su dirección.
`cartas`, **`categorias`**, **`items`**	Jerarquía del menú. `items` guarda precio (entero), puntuación 0–5, orden, visible, destacado y foto.
`usuarios`	Correo, hash bcrypt (`password_hash`), `rol` (`admin` o `cliente`), `activo`.
`comentarios`	Comentarios existentes (solo lectura en el sitio; el administrador los modera).
`sesiones`	Sesiones de servidor (connect-pg-simple).
`limite_intentos`	Contadores del límite de intentos de login, compartidos entre todas las instancias.

Las migraciones (`backend/db/migrations/`) se aplican solas al arrancar el contenedor, con un bloqueo de PostgreSQL para que dos instancias que arranquen a la vez no choquen. El catálogo de ejemplo se carga una vez si la base está vacía.

3.4 Panel de administración

El administrador entra en `/admin`. Puede crear, editar, ocultar y eliminar restaurantes, cartas, categorías y platillos, subir imágenes y moderar comentarios. "Eliminar" es **borrado lógico**: el dato deja de verse pero queda en la base. "Ocultar" lo saca del sitio y lo mantiene en el panel.

El primer administrador se crea con `ADMIN_EMAIL` y `ADMIN_PASSWORD` (mínimo 12 caracteres). En desarrollo y QA existe el usuario de demostración `admin@gmail.com` / `admin01`, que **no existe en producción**.

> **Decisión de producto**
> Los clientes no inician sesión, por lo que **no pueden comentar**. Los comentarios existentes se muestran en solo lectura y el administrador los modera.

4. Frontend

Sitio estático sin frameworks (el requisito fue no usar React). Cada pantalla es un HTML con su script:

Archivo (`frontend/public/`)	Pantalla y ruta
`carta.html` + `js/carta.js`	Carta digital: `/` y `/restaurantes/:id` (destacados, pestañas por carta, comentarios).
`carrito.html` + `js/carrito.js`	Productos seleccionados: `/carrito`.
`admin.html` + `js/admin.js`	Panel de administración: `/admin`. Aplicación de una sola página; las pantallas usan `#/…`.
`js/comun.js`	Cliente de la API (maneja el token CSRF), creación segura de elementos, formato de moneda e iconos.
`404.html`	Página de error para rutas que no existen.

4.1 Cómo se evita el XSS

- Todo dato que llega de la API se inserta como **nodo de texto** (función `h()`), nunca como HTML: un nombre o comentario con `<script>` se ve tal cual y no se ejecuta.
- Las URL de imágenes se aceptan solo si son rutas internas (`/media/…`, `/imagenes/…`); `javascript:` o `//otro-sitio` se reemplazan por la imagen por defecto.
- Las páginas no tienen scripts en línea ni manejadores `onclick`, por lo que la política `script-src 'self'` es estricta.
- La única excepción de `innerHTML` son los iconos SVG, que son constantes del propio archivo.

4.2 Nginx

Ruta	Comportamiento
`/`, `/restaurantes/<id>`, `/carrito`, `/admin`	Entrega la página estática correspondiente. El panel se sirve con `no-store` y `noindex`.
`/css`, `/js`	Se revalidan siempre (un despliegue nuevo se ve de inmediato).
`/img`, `/imagenes`, `/vendors`	Caché de 7 días.
`/api/`, `/media/`, `/salud`	Se reenvían a la API (`back:3000`) con `X-Forwarded-For` y `X-Forwarded-Proto`.
Cualquier otra	404 con la página de error.

Las páginas llevan Content-Security-Policy (`default-src 'self'`, `script-src 'self'`, `frame-ancestors 'none'`…), `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy` y `Permissions-Policy`. La imagen usa `nginx-unprivileged` (corre sin privilegios, puerto 8080).

5. Seguridad implementada

5.1 Hallazgos del informe de pentesting y cómo se resuelven

El informe original de la versión PHP (VUL-01 a VUL-22) deja de aplicar por diseño:

Hallazgo	En esta versión
SQL Injection (VUL-02/03/04/05)	Solo consultas parametrizadas en `models/`; los ids de la URL se validan como enteros positivos (404 si no lo son).
XSS reflejado y almacenado (VUL-06/07/08)	El frontend inserta los datos como texto; CSP `script-src 'self'` y sin scripts en línea.
Control de acceso (VUL-09, 20)	`requiereAdmin` (rol) en el servidor en cada petición; filtros `eliminado IS NULL`.
Cookie y sesión (VUL-10, 15)	`HttpOnly`, `SameSite=Lax`, `Secure` en producción; sesión nueva al iniciar sesión; el logout la destruye.
Cabeceras y errores (VUL-11/12/13)	`helmet` y CSP; sin `X-Powered-By`; manejador global que no filtra detalles internos.
CSRF (VUL-14)	Token por sesión en la cabecera `X-CSRF-Token`, comparado en tiempo constante.
Fuerza bruta (VUL-16)	5 intentos fallidos por IP + correo en 15 minutos, contador global en PostgreSQL; respuesta genérica (429).
Contraseñas en texto plano (VUL-17)	bcrypt de 12 rondas; comparación en tiempo constante y hash ficticio para igualar tiempos.
Credenciales de BD (VUL-19)	Solo por variables de entorno, distintas por ambiente.
Archivos expuestos (VUL-01)	Nginx solo sirve `frontend/public/`; el código y el SQL viven en otro contenedor sin puertos.
Dependencias (VUL-18)	Versiones actuales, `npm audit` y `eslint-plugin-security` en cada ejecución; auditoría de producción en 0.
Acceso a datos (VUL-21) y rutas de imágenes (VUL-22)	Capa única de modelos; el id de la ruta es siempre un entero validado.

5.2 Controles del panel y de las imágenes

Control	Cómo funciona
Autorización por rol	Sin sesión → 401; con otro rol → 403. Se comprueba siempre en el servidor; el frontend solo muestra u oculta pantallas.
Login seguro	Mensaje genérico ante cualquier fallo, igualación de tiempos, límite de intentos y sesión nueva (y token CSRF nuevo) al entrar. Sesión de administrador de 2 horas.
Validación de entrada	Esquemas con lista blanca de campos (tipo, largo y rango). Los campos no declarados se ignoran: no hay asignación masiva.
Subida de imágenes	Tipo real por **firma** (JPG, PNG o WEBP; SVG rechazado), máximo 2 MB, nombre aleatorio de 32 caracteres, no se escribe nada antes de validar.
Entrega de imágenes	`X-Content-Type-Options: nosniff` y `Content-Security-Policy: sandbox`; solo se sirven claves con el formato exacto (sin recorrido de rutas).
Respuestas de la API	`Cache-Control: no-store` y `X-Robots-Tag: noindex`: dependen de la sesión y no deben quedar en cachés.

Control	Cómo funciona
Almacén S3	Bucket privado: bloqueo de acceso público, cifrado AES-256, política que exige HTTPS y credenciales por rol de instancia (IMDSv2).

5.3 Contenedores y red

- La API **no publica puertos**: solo el Nginx de la misma instancia la alcanza por una red interna de Docker.
- Contenedores en producción: sistema de archivos de solo lectura, `cap_drop: ALL`, `no-new-privileges`, límites de memoria y de procesos, registros rotados.
- Base de datos en subredes privadas, sin acceso público, con conexión TLS verificada contra el certificado raíz de Amazon (incluido en la imagen).
- Cadena de grupos de seguridad de mínimo privilegio: Internet → balanceador (80/443) → instancia (80, solo desde el balanceador) → RDS (5432, solo desde la instancia).

6. Protección de /admin: defensa en profundidad

Un panel de administración se protege en **capas**: si una falla, la siguiente sigue en pie. Hay siete capas; las cuatro de la aplicación están implementadas y probadas, las tres de Cloudflare están documentadas pero no aplicadas.

#	Capa	Dónde	Estado
1	Contraseña con bcrypt, error genérico, tiempos igualados	Aplicación	✓ Implementada
2	Rol admin en el servidor, CSRF, sesión de 2 h, cookie HttpOnly + Secure + SameSite	Aplicación	✓ Implementada
3	Límite de intentos de login (5 por IP y correo en 15 min, en PostgreSQL)	Aplicación	✓ Implementada
4	Respuestas con `no-store` y `noindex`	Aplicación / Nginx	✓ Implementada
5	Regla WAF: solo ciertas IP llegan a `/admin` y `/api/admin`	Cloudflare	⛔ No aplicada
6	Rate limiting en el borde (Cloudflare corta los intentos antes de AWS)	Cloudflare	⛔ No aplicada
7	Cloudflare Access: código por correo antes de ver el login	Cloudflare	⛔ No aplicada

6.1 Por qué importan las capas 5, 6 y 7

- **Reducen la superficie de ataque.** Sin ellas cualquiera en Internet puede llegar al formulario de login y probar contraseñas; con ellas ni siquiera lo ve. Un fallo futuro en la aplicación (una vulnerabilidad nueva, una dependencia con CVE) no sería explotable desde fuera.
- **Cortan el abuso donde es más barato.** La capa 3 detiene la fuerza bruta, pero la petición ya consumió CPU y base de datos de AWS. Con la capa 6, Cloudflare absorbe el tráfico malicioso sin gastar crédito de AWS (relevante ante una denegación de servicio).

- **Añaden un segundo factor independiente de la aplicación.** Con Access hacen falta dos cosas distintas: acceso al correo autorizado y usuario/contraseña. Es como un 2FA sin escribir código.
- **Aplican el principio de defensa en profundidad** (ISO/IEC 27034, OWASP ASVS V2 y V4): ningún control es infalible, por eso se apilan.

6.2 Por qué no están aplicadas (decisión consciente)

- **La regla WAF por IP (5) no es viable:** el sitio se administra desde dos redes distintas (casa y universidad), con IP diferentes y dinámicas. Una regla fija bloquearía al propio administrador cada vez que cambie la IP, un riesgo real en plena demostración.
- **Access (7) sí resolvería el problema de la IP**, pero exige activar Zero Trust (a veces pide una tarjeta aun en el plan gratuito) y depende de la cuenta de Cloudflare del alumno; en un laboratorio con cuentas de AWS rotativas se consideró un costo de gestión innecesario.
- **El rate limiting de borde (6)** en el plan gratuito permite una sola regla y duplica el límite que ya aplica la aplicación (capa 3).
- Las capas 1 a 4 **ya cubren el riesgo principal** (adivinar contraseñas, escalar privilegios, CSRF, secuestro de sesión) y están probadas con pruebas automáticas.

> **Conclusión**
> Las capas 5 a 7 son una **mejora recomendada, no un requisito**. En un entorno real (IP fija de oficina o VPN corporativa) se activarían la 5 y la 7 de inmediato.

6.3 Cómo se activarían (para cuando se necesite)

Regla WAF por IP (gratis, 2 minutos)

1. Cloudflare → la zona → **Security** → **WAF** → **Custom rules** → **Create rule**.
2. Pulsar **Edit expression** y pegar (cambiando la IP por la propia):

```
((starts_with(http.request.uri.path, "/admin") or starts_with(http.request.uri.path,
"/api/admin"))
 and ip.src ne 203.0.113.10)
```

Acción **Block** y **Deploy**. Se bloquean **las dos rutas**: la página (`/admin`) y la API del panel (`/api/admin`); proteger solo la página dejaría la API accesible.

Cloudflare Access (sirve con IP dinámica)

1. Cloudflare → **Zero Trust** (crear el "team name" y elegir el plan Free).
2. **Access** → **Applications** → **Add an application** → **Self-hosted**, dominio `cartapnk.xyz`, ruta `admin`; agregar una segunda entrada con la ruta `api/admin`.
3. Política **Allow** → **Emails**: los correos autorizados. Método de acceso: One-time PIN.

Rate limiting en el borde

Security → **WAF** → **Rate limiting rules**: regla sobre `POST /api/admin/login` (por ejemplo 5 peticiones por minuto por IP, acción Block).

7. Ambientes y Docker

Un mismo código con distinta configuración. Cada ambiente usa un nombre de proyecto de Compose propio (`pnk-dev`, `pnk-qa`, `pnk-prod`), por lo que tiene su red y su volumen de datos.

	Desarrollo	QA	Producción
`APP_ENV`	`development`	`qa`	`production`
Imágenes Docker	API con nodemon; frontend con archivos montados	API `produccion` + contenedor de pruebas	API `produccion` + frontend Nginx
Datos	Catálogo + usuarios demo	Catálogo + usuarios demo	Catálogo; **sin** usuarios demo; administrador por variables
Secretos	Valores por defecto permitidos	Obligatorios	Obligatorios (≥ 32 caracteres)
Cookie Secure / HSTS	No	No (HTTP local)	Sí (HTTPS)
Puertos	Sitio 3000, API 127.0.0.1:3100, BD 127.0.0.1:5432	Sitio 3001	Sitio solo en 127.0.0.1 (o 80); **API y BD sin puertos**
Endurecimiento	—	—	Solo lectura, `cap_drop ALL`, límites de memoria/PIDs, logs rotados

7.1 Puesta en marcha

Requisitos: Docker con Compose v2 y Node 22+ (solo para el script que genera los archivos `env`).

```
node tools/init-env.js dev qa      # crea env/dev.env y env/qa.env con secretos aleatorios
npm run docker:dev                 # DESARROLLO  → http://localhost:3000
npm run docker:qa                  # QA          → lint + pruebas + auditoría
npm run docker:qa:app              # QA (sitio)  → http://localhost:3001
npm run docker:prod                # PRODUCCIÓN
```

AWS CloudShell

CloudShell trae Docker pero no el plugin `docker compose`. Se instala una vez:

```
mkdir -p ~/.docker/cli-plugins && curl -SL
https://github.com/docker/compose/releases/download/v2.29.7/docker-compose-linux-x86_64 -o
~/.docker/cli-plugins/docker-compose && chmod +x ~/.docker/cli-plugins/docker-compose
```

7.2 Variables de entorno principales

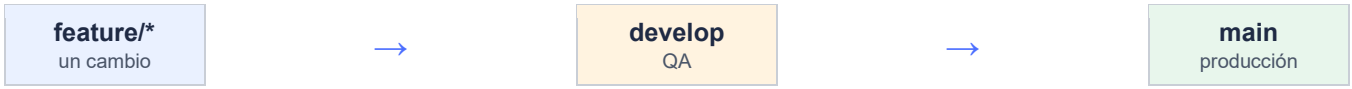
Variable	Uso
`APP_ENV`	`development`, `qa` o `production`; cambia los controles de arranque.
`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_SSL`	Conexión a PostgreSQL (`DB_SSL=true` con RDS).
`SESSION_SECRET`	Firma de la cookie de sesión (≥ 32 caracteres en QA y producción).

Variable	Uso
`COOKIE_SECURE`, `TRUST_PROXY`	Cookie Secure (HTTPS) y número de proxys de confianza (para la IP real del visitante). Compose: 1; AWS: 3 (Nginx + balanceador + Cloudflare).
`SEED_CATALOGO`, `SEED_USUARIOS_DEMO`	Carga del catálogo de ejemplo y de usuarios de demostración.
`ADMIN_EMAIL`, `ADMIN_PASSWORD`	Primer administrador (contraseña ≥ 12 caracteres).
`ADMIN_SESION_HORAS`	Duración de la sesión de administrador (por defecto 2).
`STORAGE_DRIVER`, `S3_BUCKET`, `AWS_REGION`, `UPLOADS_DIR`	Dónde se guardan las imágenes: `local` (disco) o `s3`.

8. Flujo de Git, integración continua y Jenkins

8.1 Ramas y ambientes

El código vive en un solo lugar (GitHub). Los ambientes no se pasan código entre sí: cada uno hace `git pull` de la rama que le corresponde y la ejecuta con su propia configuración.



Rama	Para qué sirve
`feature/<nombre>`	Un cambio concreto. Se abre un Pull Request hacia `develop`.
`develop`	Cambios integrados, listos para probar en QA.
`main`	Solo código aprobado; es lo que se despliega a producción.

Pendiente manual: activar la protección de `main` en GitHub (*Settings → Branches*): exigir Pull Request y que pasen los chequeos `calidad`, `frontend` e `infraestructura`.

8.2 Integración continua (GitHub Actions)

Trabajo	Qué hace
`calidad`	Levanta PostgreSQL, instala el backend y ejecuta lint (ESLint + eslint-plugin-security), las 136 pruebas y `npm audit` de producción.
`frontend`	Ejecuta las 16 pruebas con jsdom y valida la configuración de Nginx construyendo la imagen y corriendo `nginx -t`.
`infraestructura`	Valida la plantilla de CloudFormation con `cfn-lint` y revisa los scripts con ShellCheck (informativo).

8.3 Plan de Jenkins con OWASP y MITRE (4.ª entrega)

> **Estado**
> El diseño y el `Jenkinsfile` están escritos y apuntan a la estructura actual (`back` y `front`), pero **no se han ejecutado** contra un Jenkins real. Las partes que dependen de la versión de cada plugin están marcadas en el archivo.

#	Etapa	Herramienta	Qué detecta
1	Secretos	Gitleaks	Claves y contraseñas en el repositorio
2	SAST	Semgrep (OWASP Top 10, Node) + ESLint security	Inyecciones, XSS, uso inseguro de APIs
3	SCA	npm audit + OWASP Dependency-Check	Dependencias con CVE conocidos
4	Calidad	Jest + Supertest en Docker (QA)	Regresiones funcionales y de seguridad
5	Imagen	Trivy	CVE en el sistema base y las librerías
6	DAST	OWASP ZAP (baseline) contra QA	Cabeceras, cookies, XSS/SQLi reflejados
7	MITRE	`backend/tools/mapear-mitre.js`	Consolida los hallazgos por CWE → CAPEC → ATT&CK
8	Aprobación	`input` de Jenkins	Un humano autoriza producción
9	Despliegue	SSH + Docker Compose	Prueba de humo `/salud` por Cloudflare

MITRE no tiene un "plugin" para Jenkins: aporta **catálogos** que las herramientas ya usan (CVE, CWE, CAPEC, ATT&CK) y el pipeline los consolida en un informe `mitre.md`. OWASP aporta el estándar (Top 10, ASVS) y las herramientas Dependency-Check y ZAP.

9. Despliegue en AWS

9.1 Qué se crea

Todo el despliegue es **infraestructura como código** (`infra/pnk-stack.yaml`, CloudFormation) y se maneja con un solo script. No crea recursos IAM: usa el perfil de instancia existente de AWS Academy (`LabInstanceProfile`).

Recurso	Detalle
VPC `10.20.0.0/16`	2 subredes públicas (balanceador e instancias) y 2 privadas (solo RDS); sin NAT Gateway.
RDS PostgreSQL 17	`db.t3.micro`, 20 GB gp3 cifrado, en subredes privadas, sin acceso desde Internet.
Application Load Balancer	Recibe HTTP :80 (y HTTPS :443 con el certificado); comprueba `/salud` cada 15 s y saca del servicio a la instancia que no responde.
Auto Scaling Group	Mínimo 2, máximo 4 instancias `t3.small` en 2 zonas; escala por peticiones por instancia (> 300/min) y por CPU (> 55 %); reemplaza instancias enfermas; actualiza con *rolling update*.
Cada instancia	Dos contenedores en una red interna: `front` (Nginx, publicado en el puerto 80) y `back` (API, sin puertos).
Amazon S3	Bucket privado para las imágenes del panel; lo comparten todas las instancias.
Amazon ECR	Repositorio `cartapnk` con dos imágenes: `back-<etiqueta>` y `front-<etiqueta>`.
ACM	Certificado del dominio, validado por DNS.

Las instancias **no guardan estado**: las sesiones, el carrito y los contadores del límite de login viven en PostgreSQL y las imágenes en S3; por eso se pueden añadir o quitar instancias sin que nadie pierda su sesión.

9.2 Comandos de `scripts/desplegar.sh`

Comando	Qué hace
`verificar`	Comprueba herramientas y permisos de la cuenta; no crea nada.
`up [--sin-build]`	Genera los secretos, construye las dos imágenes, las sube a ECR y crea o actualiza toda la infraestructura. Espera la salud del balanceador e imprime la URL.
`status` · `url`	Estado de la pila, instancias, salud del balanceador y RDS · URL pública.
`certificado <dominio>`	Pide el certificado de ACM; si hay token de Cloudflare crea solo el registro de validación y espera a que se emita.
`dominio [<dominio>]`	Imprime qué configurar en Cloudflare con los valores de la cuenta actual y comprueba si el dominio ya funciona.
`cloudflare [<dominio>] [--reemplazar]`	Crea o corrige los registros DNS y aplica los ajustes de la zona mediante la API de Cloudflare.
`tunel`	Publica la URL con un túnel temporal de Cloudflare (sin dominio).
`carga`	Genera tráfico para demostrar el Auto Scaling.
`down [--ecr]`	Destruye todo (pide confirmación). Vacía el bucket S3 antes de borrar la pila.

9.3 Desplegar por primera vez

1. Copiar las credenciales de AWS Academy (**AWS Details** → **AWS CLI** → **Show**) a `~/.aws/credentials` y configurar la región `us-east-1`.
2. `git clone https://github.com/Alkr4/CartaPnk.git && cd CartaPnk`
3. `./scripts/desplegar.sh verificar` (todo debe salir con ✓).
4. `./scripts/desplegar.sh up` (~15 minutos; casi todo es esperar a RDS).
5. `./scripts/desplegar.sh status` y abrir la URL del balanceador.

> **Costo y tiempo**
> La infraestructura consume crédito del laboratorio mientras exista (2 instancias, RDS y balanceador). **Destruirla al terminar con `down`.** Las credenciales de AWS Academy caducan a las ~4 horas.

9.4 Demostrar el Auto Scaling

```
# Terminal 1: genera carga sobre la API
./scripts/desplegar.sh carga
# Terminal 2: observa cómo crece el grupo (tarda 3-5 minutos en reaccionar)
./scripts/desplegar.sh status
```

9.5 Actualizar la aplicación

Tras un `git pull` con cambios, `./scripts/desplegar.sh up` construye imágenes nuevas, las sube y CloudFormation **reemplaza las instancias de a una**, sin cortar el servicio. Para reutilizar las últimas imágenes sin reconstruir: `up --sin-build`.

10. Dominio, HTTPS y Cloudflare

El dominio `cartapnk.xyz` se compró en Nameslink y su DNS se delegó a **Cloudflare** (plan gratuito), que actúa como DNS, CDN y WAF. El tráfico va cifrado de punta a punta: Navegador → Cloudflare (HTTPS) → balanceador (HTTPS con certificado de ACM) en modo **Full (strict)**.

10.1 Pasos que se hacen una sola vez (manuales)

1. Comprar el dominio.
2. Agregarlo a Cloudflare (plan Free) y copiar los dos nameservers que asigna.
3. En el registrador (Nameslink), **reemplazar** sus nameservers por los de Cloudflare. Cuando el dominio aparece como **Active**, está listo.

10.2 Pasos por cada cuenta de AWS

Los certificados de ACM **no pasan de una cuenta de AWS a otra**; con una cuenta nueva hay que pedir uno nuevo y el DNS del balanceador es distinto.

Paso	Comando	Qué ocurre
1	`./scripts/desplegar.sh certificado cartapnk.xyz`	Pide el certificado y muestra el CNAME de validación (con token de Cloudflare lo crea solo). Espera hasta que quede **emitido**.
2	`./scripts/desplegar.sh up`	Activa HTTPS en el balanceador (listener 443) y la cookie `Secure`.
3	`./scripts/desplegar.sh cloudflare cartapnk.xyz`	Apunta el dominio al balanceador nuevo y aplica los ajustes de la zona. Sin token, `dominio` imprime lo que hay que crear a mano.

10.3 Qué configura el comando cloudflare

Acción	Detalle
Registro de validación del certificado	CNAME hacia ACM, *DNS only* (nube gris); solo si el certificado aún no está emitido.
CNAME del sitio	`cartapnk.xyz` → DNS del balanceador, *Proxied* (nube naranja). Si ya existe, lo **actualiza** (el DNS del balanceador cambia con cada pila nueva).
Ajustes de la zona	SSL/TLS **Full (strict)**, *Always Use HTTPS*, TLS mínimo **1.2**, *Rocket Loader* apagado (inyecta scripts y la política CSP los bloquearía).
Verificación	Pide `https://dominio/salud` y explica el resultado (200, 522, 525…).

El comando es **idempotente** (se puede repetir). Si existe un registro de otro tipo con el mismo nombre (por ejemplo un `A` de "estacionamiento" del registrador) se detiene y lo informa; con `--reemplazar` lo borra y crea el CNAME. Para dominios de sufijo compuesto (`midominio.co.uk`) se indica la zona con `PNK_CF_ZONA`.

10.4 Token de la API de Cloudflare

1. Cloudflare → perfil → **My Profile** → **API Tokens** → **Create Token** → **Create Custom Token**.
2. Permisos: `Zone · DNS · Edit`, `Zone · Zone Settings · Edit` y `Zone · Zone · Read`.
3. Recursos: **Specific zone** → **cartapnk.xyz** (así el token no puede tocar ninguna otra zona). Conviene fijar una fecha de vencimiento.
4. Copiar el valor (se muestra **una sola vez**) y usarlo con `export CLOUDFLARE_API_TOKEN=...`.

> **Seguridad del token**
> Es una credencial: quien la tenga puede cambiar el DNS del dominio. Por eso se limita a **una zona**, con los **permisos mínimos** y vencimiento.
> Viaja solo por variable de entorno y por la entrada estándar de `curl` (`-K -`): **no aparece en la lista de procesos ni en el historial**, y nunca se escribe en el repositorio.
> Si se filtrara: Cloudflare → API Tokens → **Roll** (cambia el valor) o **Delete**.

10.5 Diagnóstico de errores de Cloudflare

Síntoma	Causa probable	Qué hacer
Error **522 / 523 / 524**	El CNAME apunta a un balanceador que ya no existe o no responde	`./scripts/desplegar.sh cloudflare` (o editar el destino con el DNS nuevo)
Error **525 / 526**	SSL de Cloudflare no está en Full (strict) o el balanceador aún no usa el certificado	Revisar el modo SSL y ejecutar `up --sin-build`
El dominio no responde	El DNS aún no se propaga, o el sitio no está desplegado	Esperar unos minutos y ejecutar `./scripts/desplegar.sh dominio`
"Esta cuenta de AWS no tiene un certificado…"	Cuenta nueva	`./scripts/desplegar.sh certificado cartapnk.xyz`

11. Operación y recuperación ante desastres

11.1 Qué ocurre si se borra algo

Si se borra…	Qué ocurre	Qué hacer
Una instancia EC2	Nada visible: el Auto Scaling crea otra sola en 3 a 5 minutos y la otra atiende mientras tanto. Sesiones y datos no dependen de la instancia.	Nada.
Todas las instancias	El sitio cae unos minutos y el grupo las recrea solo.	Esperar; revisar con `status`.
El equipo desde donde se ejecuta el script (CloudShell, EC2 de trabajo)	La producción no se toca. Se pierde la carpeta `.despliegue/` (secretos).	Volver a clonar el repositorio.
La pila de CloudFormation	Se borra red, balanceador, instancias, **RDS (con sus datos)** y **el bucket S3 (con sus imágenes)**. El sitio queda caído.	Reinstalar (sección 11.2).

Si se borra…	Qué ocurre	Qué hacer
El repositorio ECR	La infraestructura sigue, pero no podrá lanzar instancias nuevas.	`up` reconstruye y sube las imágenes.
El certificado de ACM	HTTPS deja de funcionar al renovar o recrear el balanceador.	`certificado cartapnk.xyz` y luego `up --sin-build`.
El registro DNS en Cloudflare	El dominio deja de resolver.	`cloudflare` (o crear de nuevo el CNAME).

Se recupera solo al reinstalar: el código (GitHub), el catálogo de ejemplo y el usuario administrador. **No se recupera:** los restaurantes, platillos e imágenes creados desde el panel, porque en el laboratorio la base no tiene copias automáticas y el bucket se elimina con la pila.

11.2 Reinstalación completa (~25 minutos)

Sirve igual con una cuenta de AWS nueva.

1. Copiar las credenciales de AWS (AWS Details → AWS CLI) a `~/.aws/credentials`; `aws configure set region us-east-1`; `aws sts get-caller-identity` debe mostrar la cuenta.
2. `git clone https://github.com/Alkr4/CartaPnk.git && cd CartaPnk`
3. `./scripts/desplegar.sh verificar`
4. `./scripts/desplegar.sh certificado cartapnk.xyz` (reutiliza el certificado si ya existe; en una cuenta nueva pide uno y crea su registro de validación).
5. `./scripts/desplegar.sh up`
6. `./scripts/desplegar.sh cloudflare cartapnk.xyz` (o `dominio` y crear los registros a mano).
7. Probar `https://cartapnk.xyz` y `https://cartapnk.xyz/admin`. La contraseña del administrador está en `.despliegue/secretos.env`.

11.3 Secretos

`up` genera contraseñas aleatorias (base de datos, sesión y administrador) y las guarda en `.despliegue/secretos.env` del equipo desde el que se ejecuta; no están en Git. **Conviene guardarlas en un gestor de contraseñas.** Si se pierde el archivo y la pila sigue viva, un nuevo `up` genera secretos nuevos y actualiza la pila: la contraseña del administrador cambia y no hay pérdida de datos.

11.4 Solución de problemas

Síntoma	Causa probable	Qué hacer
`ExpiredToken` / "credenciales no válidas"	Caducó la sesión del laboratorio (~4 h)	Copiar de nuevo las credenciales
`verificar` marca ✗ en algún servicio	El laboratorio no lo habilita o la sesión caducó	Reiniciar el laboratorio y repetir
La pila falla con la versión del motor o la clase de RDS	PostgreSQL 17 o `db.t3.micro` no disponibles	`PNK_DB_ENGINE_VERSION=16 ./scripts/desplegar.sh up`

Síntoma	Causa probable	Qué hacer
AccessDenied en el perfil de instancia	`LabInstanceProfile` no existe con ese nombre	`PNK_INSTANCE_PROFILE=<nombre> ./scripts/desplegar.sh up`
"Aún no hay instancias saludables"	La instancia sigue arrancando o no pudo descargar las imágenes	`status`; revisar `aws ec2 get-console-output` y buscar errores de `pnk-userdata`
`docker build` se queda en `npm ci`	Poca RAM (servidor de 1 GB) y sin swap	Crear un swap de 2 GB o usar un equipo con más memoria
`unknown shorthand flag: 'p' in -p`	CloudShell sin el plugin `docker compose`	Instalarlo (ver sección 7.1)

12. Pruebas y calidad

Capa	Herramienta	Qué verifica
Backend (136 pruebas, 7 conjuntos)	Jest + Supertest contra PostgreSQL real	Datos públicos de la carta; carrito (el precio lo define la base de datos); CSRF; cabeceras y cookies; control de acceso por rol; fuerza bruta (429); CRUD completo; validación (422) y asignación masiva; subidas hostiles (SVG, script, ejecutable, > 2 MB); `/media` sin recorrido de rutas; almacenamiento local y S3.
Frontend (16 pruebas)	`node:test` + jsdom (sin base de datos)	Renderizado de la carta, el carrito y el panel; **anti-XSS** (el HTML de la API se muestra como texto); URLs de imagen no internas; envío del token CSRF; errores 422 por campo; confirmación antes de eliminar; ausencia de scripts en línea; configuración de Nginx.
Contrato front-back	Páginas reales (jsdom) contra la API real	Login, crear restaurante y carta, 422, eliminar, carrito: comprobado manualmente durante el desarrollo.
Análisis estático	ESLint + `eslint-plugin-security` (`--max-warnings 0`)	Inyección, expresiones regulares inseguras, acceso dinámico a objetos, `eval`.
Dependencias	`npm audit --omit=dev`	Sin vulnerabilidades de producción.
Infraestructura	`cfn-lint` y ShellCheck (informativo)	Plantilla de CloudFormation y scripts.

Además de las pruebas automáticas, el ambiente QA y el despliegue en AWS se verificaron de forma manual: salud por Nginx hasta la base de datos, cabeceras de seguridad, `401` en la API del panel sin sesión y `404` en rutas desconocidas.

13. Limitaciones conocidas y trabajo pendiente

13.1 Limitaciones (laboratorio)

- **Sin copias de seguridad ni Multi-AZ** en RDS (`BackupRetentionPeriod: 0`): solo para laboratorio. En una cuenta real se activarían los backups, el versionado de S3 y Multi-AZ.

- **Instancias en subredes públicas** (para descargar imágenes sin NAT Gateway), aunque solo aceptan tráfico del balanceador.
- **Secretos en el `user-data`** de la plantilla de lanzamiento; en producción real se usaría AWS Secrets Manager con un rol que los lea.
- **Rol `LabRole`** de AWS Academy muy amplio; en una cuenta real se usaría un rol con permiso solo sobre el bucket.
- Sin WAF de AWS: el WAF de Cloudflare cumple ese papel, pero las reglas de `/admin` no están activadas (sección 6).
- Los clientes no pueden comentar (no tienen login).

13.2 Pendiente

Tarea	Detalle
Jenkins con OWASP / MITRE	4.ª entrega: ejecutar por primera vez el pipeline y ajustar a las versiones de los plugins.
Protección de la rama `main`	Manual en GitHub: exigir PR y chequeos `calidad`, `frontend` e `infraestructura`.
Fusionar ramas pendientes	`feature/comando-dominio` y `feature/cloudflare-api` hacia `develop` y luego `main`.
Evidencia de Auto Scaling	Capturas del grupo creciendo con `carga` y de los recursos en la consola de AWS, antes de destruir la infraestructura.
Apagar la infraestructura	`./scripts/desplegar.sh down` al terminar: consume crédito mientras exista.